

1.3 Analisis Operaciones de Usuario

1.3.1 Create operations

1.3.1.1 Create folder operation

- SMB2 CREATE
 - CreateOptions: Bit **FILE_DIRECTORY_FILE** □ *Ultimo bit significativo* = 1
- SMB2 CLOSE

1.3.1.2 Create file operation

- SMB2 CREATE
 - CreateOptions: Bit **FILE_DIRECTORY_FILE** □ *Ultimo bit significativo* = 0
- SMB2 CLOSE

1.3.2.1 Upload folder operation

- SMB2 CREATE
- SMB2 FIND (QUERY_DIRECTORY)
 - Como se observa en el paquete 147 de la **imagen de la operación**, el comando **Find** se ejecuta directamente sobre el recurso recién creado (**CarpetaVacía2**) y no sobre el directorio raíz. Este paso es fundamental en la operación de subida, ya que permite al cliente validar la creación del contenedor y sincronizar sus atributos antes de dar la operación por finalizada con el **CLOSE** del paquete 149
- SMB2 CLOSE

1.3.2.2 Upload file operation

- SMB2 CREATE
- SMB2 WRITE (Uno o multiples concatenados) (0x0009)
 - **Offset**: La request inicial del WRITE tiene que tener *Offset: 0*
 - **Length**: El tamaño total del archivo subido es la suma de los campos Length de todas las solicitudes WRITE asociadas con el mismo File ID (GUID).
- SMB2 CLOSE

1.3.2.3 Download folder operation

- SMB2 CREATE
- SMB2 FIND (QUERY_DIRECTORY)
- SMB2 CLOSE

1.3.2.4 Download file operation

Posibles causas de la falta de visibilidad de los paquetes durante el análisis.

- **Optimización del sistema:** Una posibilidad es que el sistema operativo, al detectar que el origen y el destino de los datos residen en el mismo almacenamiento físico, gestione la transferencia de forma interna mediante la memoria caché. Esto evitaría que la solicitud de datos llegara a atravesar la pila de red, haciendo el tráfico invisible para el analizador.
- **Velocidad del enlace local:** Otra causa probable es que, al utilizar una red interna (*loopback*), la transferencia de información ocurre a una velocidad tan elevada que el software de captura no es capaz de procesar o registrar los mensajes individuales. La rapidez del intercambio en la propia máquina podría estar provocando una pérdida de visibilidad de las ráfagas de paquetes.
- **Complejidad del entorno:** Determinar la causa exacta requeriría un análisis de bajo nivel excesivamente avanzado que excede el alcance de este estudio, ya que intervienen múltiples capas del software de red y del sistema de archivos.

Detalles técnicos observados en entorno remoto:

- **Flujo de solicitud:** Se documentó que la lectura es una operación iniciada por el cliente, quien envía un paquete indicando el *Offset* (posición de inicio) y el *Length* (cantidad de bytes solicitados).
- **Segmentación de datos:** En este entorno, Wireshark sí registró cómo los archivos son fragmentados. El servidor responde con múltiples paquetes de datos hasta completar la petición, permitiendo analizar el comportamiento del protocolo en condiciones de latencia real.

Estructura de la operación:

- SMB2 CREATE
- SMB2 READ (Uno o multiples concatenados) (0x0008)
 - **Offset:** Suele empezar en 0 para una descarga de archivo completa e incrementa secuencialmente.
 - **Length:** La suma de todos los campos de longitud en las respuestas del READ es igual a la cantidad total de datos transferidos.
- SMB2 CLOSE

En la fase de diseño del experimento, la propuesta inicial para validar la operación de lectura consistía en realizar la subida de un archivo directamente a la carpeta física del equipo servidor (PC A). Bajo esta premisa, se esperaba que la aparición del archivo en el recurso compartido provocará una respuesta automática de sincronización hacia el equipo cliente (PC B), permitiendo capturar el tráfico de red resultante.

Sin embargo, durante la ejecución se observó que este enfoque no generaba la actividad esperada en Wireshark. Como se ha podido contrastar, SMB2 no funciona como un sistema de sincronización activa (estilo Dropbox), sino como un protocolo de **acceso bajo demanda**. Al añadir un archivo en el

servidor, el cliente no recibe información de forma proactiva si no existe una petición previa, lo que invisibilizaba la operación de lectura en el analizador de protocolos. La modificación local en el servidor no es notificada al cliente. SMB2 carece de mecanismos push de sincronización automática.

Cambio de metodo

Para resolver esta falta de visibilidad y validar con éxito el flujo de información desde el servidor hacia el cliente, se redefinió la metodología utilizando el comando : **copy "Z:\ejemplo.txt" "C:\carpetaEjemplo"** en el cliente.

La unidad virtual **Z:** funciona como un mapeo de red donde el **Nodo B** actúa exclusivamente como consumidor. Dado que los archivos no residen físicamente en el cliente, cualquier operación de lectura u obtención de datos obliga a que la carga útil atraviese el **puerto 445**. Esto transforma una acción lógica del usuario en un flujo físico de paquetes, permitiendo su captura y análisis mediante **Wireshark**.

Este cambio de estrategia, ejecutado desde el equipo B utilizando la unidad virtual **Z:**, resultó ser el método óptimo para el análisis por los siguientes motivos:

- **Forzado de la transferencia:** Al iniciar la copia desde el cliente, se obliga al sistema operativo a solicitar los datos al servidor. Esto rompe la pasividad del protocolo y genera una secuencia real de comandos **READ (0x08)** que el analizador puede registrar íntegramente.
- **Simetría técnica:** Aunque el archivo se pida desde el cliente en lugar de enviarse desde el servidor, el flujo de bits es idéntico: los datos viajan desde el almacenamiento remoto hacia la memoria local. Por tanto, la simulación de "descarga" es técnicamente perfecta y representativa de un entorno de red corporativo.

Conclusión del cambio metodológico

La transición desde una *subida pasiva* en el servidor hacia una *descarga activa* desde el cliente permitió superar las barreras de optimización y caché de metadatos de Windows. Este enfoque no solo facilitó la captura de la carga útil del archivo, sino que validó la naturaleza bajo demanda del protocolo SMB2, proporcionando una base de datos mucho más rica y detallada para el estudio de los tiempos de respuesta y la fragmentación de paquetes.

Validación técnica del flujo de datos bajo demanda

Tras las pruebas realizadas, se ha verificado que el protocolo SMB2 en Windows no realiza una sincronización automática de archivos, sino un **acceso remoto bajo demanda**. Esta distinción técnica es fundamental para entender la visibilidad de los mensajes en el analizador de tráfico.

Relación entre operaciones del sistema y mensajes SMB2: Tal y como se ha contrastado, el sistema operativo utiliza diferentes niveles de caché (metadatos y contenido) que pueden invisibilizar el tráfico si no se fuerza una acción explícita. Para que el protocolo genere mensajes de red, es necesario que el cliente interactúe directamente con el recurso:

- **Apertura de archivos:** Al intentar acceder a un documento en la unidad remota, el sistema realiza llamadas equivalentes a una apertura de *handle* (como **fopen**), lo que se traduce en paquetes SMB2 de creación o apertura de sesión.
- **Transferencia de contenido:** Para visualizar los paquetes de datos, es necesario forzar una lectura del contenido (equivalente a un **fread**). Por este motivo, el uso del comando de copia desde la unidad **Z:** al equipo local ha resultado ser el método más efectivo, ya que obliga al protocolo a solicitar y transferir el flujo de bits real a través de la red para completar la operación.

Conclusión sobre el método de Descarga: El éxito en la captura de los mensajes **READ (0x08)** al descargar archivos hacia la carpeta local confirma que esta acción rompe la pasividad del acceso remoto. Al mandar el archivo desde el cliente, se provoca una secuencia de apertura, lectura y transferencia que el analizador de protocolos puede registrar íntegramente. Este enfoque no solo simula una descarga real de un usuario en una red corporativa, sino que garantiza que la carga útil del archivo atraviese la interfaz de red, permitiendo auditar la segmentación y el direccionamiento del protocolo.

1.3.3 Metadata Operations and Organization

1.3.3.1 Rename folder operation

- SMB2 CREATE
- SMB2 SET_INFO (0x0011)
 - **FILE_INFORMATION_CLASS:** [FileRenameInformation](#) (0x0A)
 - **FileName:** Dentro del cuerpo del paquete, se envía la cadena de texto con el **nuevo nombre**.
- SMB2 CLOSE

1.3.3.2 Rename file operation

- SMB2 CREATE
- SMB2 SET_INFO (0x0011)
 - **FILE_INFORMATION_CLASS:** [FileRenameInformation](#) (0x0A)
- SMB2 CLOSE

1.3.3.3 Move folder operation

- SMB2 CREATE
- SMB2 SET_INFO (0x0011)
 - **FILE_INFORMATION_CLASS:** [FileRenameInformation](#) (0x0A)
 - **FileName:** Dentro de este paquete, se indica la **ruta completa de destino** donde se desea reubicar la carpeta.
- SMB2 CLOSE

1.3.3.4 Move file operation:

- SMB2 CREATE
- SMB2 SET_INFO (0x0011)
 - **FILE_INFORMATION_CLASS:** [FileRenameInformation](#) (0x0A)
 - **FileName:** El paquete contiene la nueva ruta de acceso y el nombre del archivo en el destino. El servidor procesa esta petición y actualiza su tabla de archivos interna para reflejar el cambio.
- SMB2 CLOSE

1.3.4 Deletion and Query Operations

1.3.4.1 Delete folder operation

- SMB2 CREATE
- SMB2 SET_INFO (0x0011)
 - **FILE_INFORMATION_CLASS:** [FileDispositionInformation](#) (0x0D)
 - **DeletePending:** Se establece en **1**. Esto indica al servidor que el objeto debe ser eliminado en cuanto se cierre el manejador. [Enlace](#)
- SMB2 CLOSE

1.3.4.2 Delete file operation

- SMB2 CREATE
- SMB2 SET_INFO (0x0011) (REQUEST)
 - **FILE_INFORMATION_CLASS:** [FileDispositionInformation](#) (0x0D)
 - **DeletePending:** Se establece en **1**. Esto indica al servidor que el objeto debe ser eliminado en cuanto se cierre el manejador.
- SMB2 CLOSE

1.3.4.3 List directory operation

- SMB2 CREATE
- SMB2 FIND (QUERY_DIRECTORY) - (0x0E) (Uno o multiples concatenados)
 - **Search Pattern:** Suele incluir un patrón de búsqueda (comúnmente *****) para recuperar todos los elementos.
- SMB2 CLOSE

1.3.5 Copy and Compression Operations

1.3.5.1 Copy folder operation

- SMB2 CREATE
- SMB2 FIND (QUERY_DIRECTORY)
- SMB2 CLOSE

1.3.5.2 Copy file operation

Flujo de subida (Copia de local a red)

- SMB2 CREATE
- SMB2 SET_INFO (0x0011) (Uno o multiples según lo que se desee cambiar)
- SMB2 WRITE (Uno o multiples concatenados) (0x0009)
 - **Offset:** La request inicial del WRITE tiene que tener *Offset: 0*
 - **Length:** El tamaño total del archivo subido es la suma de los campos Length de todas las solicitudes WRITE asociadas con el mismo File ID (GUID).
- SMB2 CLOSE

Flujo de Descarga (Copia de red a local)

- SMB2 CREATE
- SMB2 READ (Uno o multiples concatenados) (0x0008)
 - **Offset:** Suele empezar en 0 para una descarga de archivo completa e incrementa secuencialmente.
 - **Length:** La suma de todos los campos de longitud en las respuestas del READ es igual a la cantidad total de datos transferidos.
- SMB2 CLOSE

1.3.5.4 Compress file operation

A. Flujo de Lectura (Archivo Origen)

- SMB2 CREATE
- SMB2 READ (Uno o multiples concatenados) (0x0008)
 - **Offset:** Suele empezar en 0 para una descarga de archivo completa e incrementa secuencialmente.
 - **Length:** La suma de todos los campos de longitud en las respuestas del READ es igual a la cantidad total de datos transferidos.
- SMB2 CLOSE

B. Flujo de Escritura (Archivo Destino)

- SMB2 CREATE
- SMB2 SET_INFO (Uno o multiples concatenados) (0x0011)
 - **FILE_INFORMATION_CLASS:** [FileAllocationInformation](#) (0x13)
- SMB2 WRITE (Uno o multiples concatenados) (0x0009)
 - **Offset:** La request inicial del WRITE tiene que tener *Offset: 0*
 - **Length:** El tamaño total del archivo subido es la suma de los campos Length de todas las solicitudes WRITE asociadas con el mismo File ID (GUID).
- SMB2 CLOSE

La operación se considera concluida tras el comando SMB2 CLOSE sobre ambos ficheros. El cierre del archivo de origen ocurre generalmente una vez que los datos han sido cargados en el buffer del cliente, mientras que el cierre del archivo **.zip** marca la consolidación de la operación en el sistema de archivos remoto.

La observación de un FileId nulo en el paquete 297 es coherente con la sección **2.2.14.1** del estándar [MS-SMB2]. Según la norma, el servidor tiene la obligación (MUST) de devolver los identificadores **Persistent** y **Volatile** únicamente como parte de la respuesta SMB2 CREATE. Dado que la estructura de respuesta para operaciones como WRITE no exige la retransmisión de estos manejadores, el protocolo omite esta información en la cabecera y delega la correlación de la operación al MessageID.

La discrepancia entre los campos **Length** de las operaciones de lectura y escritura es la evidencia física del proceso de **deflación de datos**. En la traza se observa cómo 365 bytes de información original se transforman en una carga útil de 343 bytes tras ser procesados por el cliente, confirmando que la compresión se realiza en el host local antes de transmitir la carga hacia el servidor.

1.3.6 Modification Operation

1.3.6.1 Modify file operation

A. Ciclo de Lectura y Carga en Memoria

Se identifica una fase inicial de preparación donde el cliente solicita un **SMB2 CREATE** seguido de un **SMB2 READ** (Mensajes 41-42).

- SMB2 CREATE
- SMB2 READ (Uno o multiples concatenados) (0x0008)
 - **Offset:** Suele empezar en 0 para una descarga de archivo completa e incrementa secuencialmente.
 - **Length:** La suma de todos los campos de longitud en las respuestas del READ es igual a la cantidad total de datos transferidos.
- SMB2 CLOSE

B. Fase de Escritura y Sincronización de Metadatos

Tras la acción de guardado por parte del usuario, se inicia un segundo flujo:

- SMB2 CREATE
- SMB2 SET_INFO (Uno o multiples concatenados) (0x0011)
 - **FILE_INFORMATION_CLASS:** [FileBasicInformation](#) (0x4)
 - **FILE_INFORMATION_CLASS:** [FileEndOfFileInformation](#) (0x14)
 - **FILE_INFORMATION_CLASS:** [FileAllocationInformation](#) (0x13)
- SMB2 WRITE (Uno o multiples concatenados) (0x0009)
 - **Offset:** La request inicial del WRITE tiene que tener *Offset: 0*
 - **Length:** El tamaño total del archivo subido es la suma de los campos Length de todas las solicitudes WRITE asociadas con el mismo File ID (GUID).
- SMB2 IOCTL (0x000B)
 - **FSCTL_QUERY_FILE_REGIONS:** Validación de las regiones de datos y consistencia del archivo antes del cierre.
- SMB2 CLOSE

Comando atomico para Modify

- SMB2 CREATE
- SMB2 SET_INFO (Uno o multiples concatenados) (0x0011)

- **FILE_INFORMATION_CLASS:** [FileEndOfFileInformation](#) (0x14)
- **FILE_INFORMATION_CLASS:** [FileAllocationInformation](#) (0x13)
- SMB2 WRITE (Uno o multiples concatenados) (0x0009)
 - **Offset:** La request inicial del WRITE tiene que tener *Offset: 0*
 - **Length:** El tamaño total del archivo subido es la suma de los campos Length de todas las solicitudes WRITE asociadas con el mismo File ID (GUID).
- SMB2 CLOSE